This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

本文档由 funstory.ai 的开源 PDF 翻译库 BabelDOC v0.6.2 (https://github.com/funstory-ai/BabelDOC) 翻译，本仓库正在积极的建设当中，欢迎 star 和关注。

Plenoxels：无需神经网络的辐射场

Sara Fridovich-Keil* Alex Yu*
Benjamin Recht

MatthewTancik Qinhong Chen
Angjoo Kanazawa

加州大学伯克利
分校

摘要

我们介绍了 *Plenoxels*（全光体素），一种用于光真实感视图合成的系统。*Plenoxels* 将场景表示为一个稀疏的 *3D* 网格，并使用球谐函数。这种表示可以通过梯度方法和正则化从校准图像中优化，而无需任何神经组件。在标准的基准任务上，*Plenoxels* 的优化速度比神经辐射场快两个数量级，且视觉质量没有损失。有关视频和代码，请参见*https://alexyu.net/plenoxels*。

1. 引言

近期的研究利用隐式、基于坐标的神经网络作为 3D 表示，从校准的 2D 图像超视觉中优化 3D 体积。特别是，神经辐射场（NeRF）[28]展示了光真实感的新视角合成，捕捉了场景几何以及视角相关的效果。然而，这种令人印象深刻的品质需要大量的计算时间，无论是训练还是渲染，训练时间超过一天，单 GPU 渲染每帧需要 30 秒。后续的多篇论文 [9, 10, 21, 37, 38, 59] 减少了渲染的计算成本，但单 GPU 训练仍然需要数小时，这一瓶颈限制了光真实感体积重建的实际应用。

在本文中，我们展示了如何从头开始训练一个辐射场，无需神经网络，同时保持NeRF质量并将优化时间缩短两个数量级。我们提供了一个定制的CUDA [31] 实现，利用模型的简单性来实现显著的加速。我们在单个Titan RTX GPU上的典型优化时间在边界场景中为11分钟（与NeRF的大约1天相比， 100×速度提升超过），在无边界场景中为27分钟（与至大约 4 天++ [60]，再次超过 100×速度提升）。尽管我们的实现不是针对快速渲染进行优化的，但我们可以在交互速率下（15 帧每秒）渲染新的视角）。如果需要更快的渲染速度，我们的优化 Plenoxel 模型可以转换为 PlenOctree [59]。

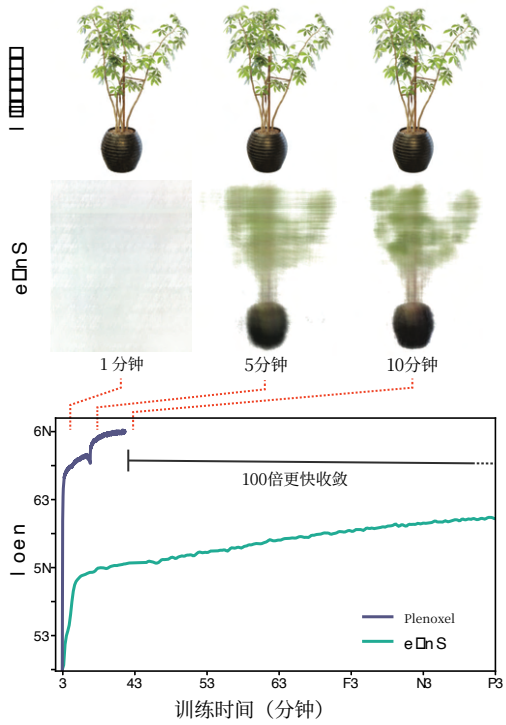


图1. **Plenoxel：全视域体积元素**用于快速优化辐射场。我们展示了直接优化一个完全显式的3D模型可以匹配现代基于神经网络的NeRF等方法的渲染质量，同时优化速度提高了两个数量级。

具体来说，我们提出了一种显式的体素表示方法，基于无需任何神经网络的视点相关稀疏体素网格。我们的模型可以渲染逼真的新视点，并从校准的2D照片中进行端到端优化，利用训练视点上的可微分渲染损失以及总变差正则化器。

* 作者们对这项工作贡献相同。

我们称我们的模型为Plenoxel，意为plenoptic volume elements（全光体积元素），因为它由一个稀疏体素网格组成，其中每个体素存储密度和球谐系数，这些系数用于模拟视点依赖性 [1]。通过插值这些系数，Plenoxels实现了全光函数的连续模型 [1]：体积内每个位置和每个方向上的光。为了在单个GPU上实现高分辨率，我们剪枝空体素并遵循由粗到细的优化策略。尽管我们的核心模型是一个有界体素网格，但我们证明可以通过使用归一化设备坐标（对于正向视角场景）或通过用多球面图像包围我们的网格来编码背景（对于 360° 场景）来建模无界场景。

我们的方法揭示了光真实感体积重建可以使用逆问题的标准工具来实现：一种数据表示、一个正向模型、一个正则化函数和一个优化器。我们的方法表明，这些组件中的每一个都可以很简单，同时仍然可以达到最先进的结果。我们的实验表明，神经辐射场的关键要素不是神经网络，而是可微分的体积渲染器。

2. 相关工作

经典体积重建。 我们从经典体积重建方法的简要概述开始，重点关注那些在我们的工作中找到应用的方法。体积渲染最常见的经典方法包括体素网格 [7, 14, 19, 22, 43–45, 53, 54] 和多平面图像 (MPIs) [27, 34, 48, 49, 58, 62]。体素网格能够表示任意拓扑结构，但在高分辨率下可能会受到内存限制。减少体素网格内存需求的一种方法是对分层结构进行编码，例如使用八叉树 [12, 40, 52, 55]（参见 [17] 的综述）；我们使用了一种更简单的稀疏数组结构。使用这些基于网格的表示形式并结合某种形式的插值 [54] 产生一个连续的表示形式，该表示形式可以使用标准信号处理方法任意调整大小（参见 [32] 的参考）。我们将这种经典采样和插值范式与 Max [24] 引入的正向体积渲染公式相结合（基于 Kajiya 和 Von Herzen [13] 的工作，并在 NeRF 中使用），直接从间接的 2D 观察中优化 3D 网格。我们进一步通过建模视点依赖性来扩展这些经典方法，我们通过优化每个体素中每个颜色通道的球谐函数系数来实现这一点。球谐函数是球面上函数的标准基，以前曾用于表示视点依赖性 [4, 36, 47, 58, 59]。

神经体积重建。 最近，神经体积重建方面取得了巨大进步，重新激发了人们对这一方向的兴趣。神经表示最初被用于建模占据 [6, 23, 26] 和符号距离 NeRF [28]在渲染一张完整的 800×800图像时，每个渲染的像素都需要在对应光线上的数百个采样位置评估基于坐标的多层感知机。许多论文随后扩展了NeRF的功能，包括在 360° 视点 [60] 中建模背景，以及为多尺度渲染 [3]加入抗锯齿。我们使用受NeRF ++ [60]启发的背景模型，将我们的Plenoxel方法扩展到无界 360° 场景中。

在这些方法中，神经体积 [22] 与我们最相似，因为它使用带插值的体素网格，但通过卷积神经网络优化该网格，并应用学习到的变形函数来提高有效分辨率（针对一个 128^3 网格）。我们证明体素网格可以直接优化，并通过剪枝和由粗到细的优化实现高分辨率，而无需任何神经网络或变形函数。

加速 NeRF。 鉴于 NeRF 在训练和渲染方面都存在巨大的计算需求，许多近期论文都提出了提高效率的方法，尤其是在渲染方面。在这些方法中，有些通过将 3D 体积细分为可更高效处理的区域来实现加速 [21, 37]。其他加速方法则专注于一系列计算和预处理或后处理方法，以消除原始 NeRF 公式中的瓶颈。JAXNeRF [8]，（JAX [5] 对 NeRF 的重新实现）通过在多个 GPU 或 TPU 上并行化，为训练和渲染都提供了加速。AutoInt [20] 重新构建基于坐标的多层感知机，以精确计算光线积分，从而在略微损失质量的情况下实现更快渲染。学习初始化 [51] 通过在许多场景上进行元学习，从更好的 MLP 初始化开始，从而在每场景数据有限时实现更快的训练和更好的先验。其他方法 [15, 29, 35] 通过预测表面或靠近表面采样来实现加速，减少了渲染每条光线所需的样本数量。

另一种方法是预训练一个 NeRF（或类似模型），然后将其提取到不同的数据结构中以支持快速推理 [9, 10, 38, 59]。特别是，PlenOc- 树 [59]将 NeRF 变体提取到稀疏体素网格中，其中每个体素使用球谐函数系数表示视点相关的颜色。由于提取的 PlenOctree 可以进一步优化，这种方法可以加快训练速度约 3×，并且因为它使用高效的 GPU 八叉树实现而不进行任何 MLP 评估，

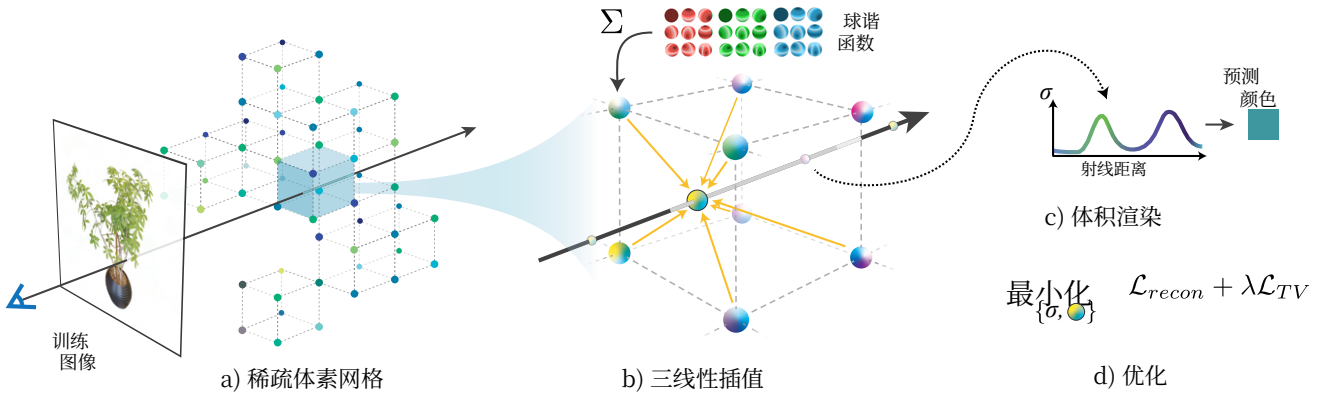


图2。 **我们稀疏Plenoxel模型的概述**。 给定一组物体或场景的图像，我们优化一个(a)稀疏体素（Plenoxel）网格，每个体素存储密度和球谐函数系数。为了渲染一条光线，我们(b)通过邻近体素系数的三线性插值计算每个采样点的颜色和不透明度。我们使用(c)可微体积渲染来积分这些样本的颜色和不透明度，借鉴了NeRF最近的成功经验 [28]。体素系数然后可以使用相对于训练图像的标准均方误差重建损失，以及总变差正则化器进行(d)优化。

它实现了 $>3000\times$ 渲染速度提升。我们的方法扩展了 PlenOctree，以对使用球谐函数的稀疏体素表示进行端到端优化，提供更快的训练速度（与 NeRF 相比，速度提升两个数量级）。我们的 Plenoxel 模型是 PlenOctree 的一般化，以支持任意分辨率（不一定是 2 的幂）的稀疏全光体素网格，并具有执行三线性插值的能力，这在稀疏体素结构中更容易实现。

3. 方法

我们的模型是一个稀疏体素网格，每个占用的体素角存储一个标量密度 σ 和每个颜色通道的球谐函数（SH）系数向量。从现在起，我们称这种表示为 *Plenoxels*。任意位置和观察方向上的密度和颜色是通过三线性插值邻近体素存储的值并评估适当观察方向上的球谐函数来确定的。给定一组校准图像，我们直接使用训练光线上的渲染损失来优化我们的模型。我们的模型如图2所示，并在下方详细描述。2。

3.1. 体积渲染

我们使用与 NeRF 相同的可微模型进行体渲染，其中通过沿光线积分采样来近似光线的颜色：

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^{N} T_i \big(1 - \exp(-\sigma_i \delta_i)\big)\mathbf{c}_i \qquad (1)$$

$$\text{where} \qquad T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right) \qquad (2)$$

T_i 表示光线$\mathbf{r}$通过采样 i时透射的光量，$(1-\exp(-\sigma_i\delta_i))$ 表示采样 i贡献的光量， σ_i 表示采样i的密度，而$\mathbf{c}_i$ 表示采样 i的颜色，距离 δ_i到下一个采样点。尽管这个公式并不精确（它假设单次散射 [13] 且采样 [24]之间的值是恒定的），但它具有可微性，并能够根据每个训练光线的误差来更新 3D 模型。

3.2. 基于球谐函数的体素网格

与 PlenOctrees [59], 类似，我们使用稀疏体素网格来构建我们的几何模型。然而，为了简化实现三线性插值，我们不使用八叉树作为数据结构。相反，我们存储一个密集的 3D 索引数组，该数组指向一个单独的数据数组，该数组仅包含占用体素的值。与 PlenOctrees 类似，每个占用体素存储一个标量密度σ以及每个颜色通道的球谐函数系数向量。球谐函数是定义在球面上的函数的正交基，低阶谐函数编码平滑（更符合朗伯模型）的颜色变化，而高阶谐函数编码更高频率（更具有镜面反射）的效果。样本$\mathbf{c}_i$ 的颜色简单地是每个颜色通道的这些谐函数基的加权总和，权重为相应的优化系数，并在适当的观察方向上评估。我们使用 2 阶球谐函数，这需要每个颜色通道 9 个系数，总共每个体素 27 个谐函数系数。我们使用 2 阶谐函数，因为 PlenOctrees 发现高阶谐函数带来的好处非常有限。

Plenoxel 网格使用三线性插值在整个体积中定义一个连续的plenoptic函数。这与 PlenOctrees 形成对比，后者假设密度和球谐函数系数在

每个体素内部保持不变。这种差异在成功优化体积方面被证明是一个重要因素，正如我们下面讨论的那样。所有系数（用于密度和球谐函数）都是直接优化的，没有任何特殊的初始化或使用神经网络进行预训练。

3.3. 插值

在每条射线上的每个采样点处，通过最近8个体素中存储的密度和调和系数进行三线性插值来计算密度和颜色。我们发现，三线性插值显著优于简单的最近邻插值；参见表。1。插值的好处有两方面：插值通过表示颜色和密度中的子体素变化来提高有效分辨率，并且插值产生连续函数近似，这对于成功的优化至关重要。这两种效果在表中都很明显。1：将最近邻插值Plenoxel的分辨率加倍，在固定分辨率下缩小了最近邻插值和三线性插值之间的大部分差距，但由于优化不连续模型的难度，仍然存在一些差距。实际上，我们发现与最近邻插值相比，三线性插值在学习率变化方面更稳定（我们在表。1中分别为每种插值方法调整了学习率，以提供尽可能接近每种设置的最佳数值）。

	PSNR ↑	SSIM ↑	LPIPS ↓
三线性插值, 256^3	30.57	0.950	0.065
三线性插值, 128^3	28.46	0.926	0.100
最近邻插值, 256^3	27.17	0.914	0.119
最近邻, 128^3	23.73	0.866	0.176

表 1. **消融关于插值方法**。 结果在 8 个 NeRF 合成场景上取平均。我们发现三线性插值提供了提高有效分辨率和提高优化的双重好处，因此，在分辨率 128^3 下，三线性插值优于在 256^3 下的最近邻插值。

3.4. 从粗到细

我们通过一种从粗到细的策略实现高分辨率，该策略从较低分辨率的密集网格开始，进行优化，剪除不必要的体素，通过在每个维度上将剩余体素各细分一半来细化，并继续优化。例如，在合成情况下，我们以 256^3 分辨率开始，并上采样到 512^3。我们使用三线性插值在每个体素细分步骤后初始化网格值。事实上，我们可以使用三线性插值在任意分辨率之间调整大小。体素剪除使用PlenOctrees [59], 中的方法执行，该方法将阈值应用于每个体素在所有训练光线上的最大权重 $T_i(1-\exp(-\sigma_i\delta_i))$（或者，或者，应用于每个体素中的密度值）。由于三线性插值，盲目剪除可能会对近表面的颜色和密度产生不利影响，因为这些点的值会与紧邻外部的体素进行插值。为了解决这个问题，我们执行一个膨胀操作，以便只有当自身及其邻居都被认为未被占用时，体素才会被剪除。

由于三线性插值，盲目剪除可能会对近表面的颜色和密度产生不利影响，因为这些点的值会与紧邻外部的体素进行插值。为了解决这个问题，我们执行一个膨胀操作，以便只有当自身及其邻居都被认为未被占用时，体素才会被剪除。

3.5. 优化

我们使用均方误差（MSE）对渲染的像素颜色进行优化，并对体素密度和球谐系数应用总变分（TV）正则化 [41]。具体来说，我们的基础损失函数是：

$$\mathcal{L} = \mathcal{L}_{recon} + \lambda_{TV}\,\mathcal{L}_{TV} \tag{3}$$

在均方误差重建损失 $\mathcal{L}_{recon}$ 和总变差正则化器 $\mathcal{L}_{TV}$ 处：

$$\mathcal{L}_{recon} = \frac{1}{|\mathcal{R}|}\sum_{\mathbf{r}\in\mathcal{R}} \|C(\mathbf{r}) - \hat{C}(\mathbf{r})\|_2^2$$

$$\mathcal{L}_{TV} = \frac{1}{|\mathcal{V}|}\sum_{\substack{\mathbf{v}\in\mathcal{V}\\ d\in[D]}} \sqrt{\Delta_x^2(\mathbf{v},d) + \Delta_y^2(\mathbf{v},d) + \Delta_z^2(\mathbf{v},d)}$$

使用 $\Delta_x^2(\mathbf{v}\ d)$ 简写表示体素 $\mathbf{v} := (i,j,k)$ 中第 d 个值与体素 $(i+1,j,k)$ 中第 d 个值之间的平方差，并除以分辨率进行归一化，类似地用于 $\Delta_y^2(\mathbf{v}\ d)$ 和 $\Delta_z^2(\mathbf{v}\ d)$，其中 D 是每个体素中存储的密度和球谐（SH）系数的总数。在实际应用中，我们为 SH 系数和 σ 值使用不同的权重。这些权重对每种场景类型（有界、正向视角和 360°）是固定的。

为了加快迭代速度，我们使用射线 $\mathcal{R}$ 的随机样本来评估 MSE 项，并使用体素 $\mathcal{V}$ 的随机样本来评估每个优化步骤中的 TV 项。我们使用与 JAXNeRF 和 Mip-NeRF [3, 8], 相同的学习率计划，但分别调整密度和调和系数的初始学习率。在主要实验中，所有数据集的所有场景中的学习率都是固定的。

直接优化体素系数是一个具有多方面挑战性的问题：需要优化的值很多（该问题是高维的），由于渲染公式，优化目标是非凸的，并且目标条件很差。条件差通常最好通过使用二阶优化算法（例如，如 [30] 中所推荐的）来解决，但这对于高维优化问题在实践中很难实现，因为Hessian太大，难以在每个步骤中轻松计算和求逆。相反，我们使用 RMSProp [11] 来缓解条件差问题，而无需二阶方法的全计算复杂度。

3.6. 无界场景

稍加修改，Plenoxels即可扩展到真实、无界的场景，包括正向视角和360°。对于正向视角场景，我们使用原始NeRF论文中定义的标准化设备坐标。



图3。 **对TV正则化的消融**。 在无TV的正向视角场景中，可见明显的伪影，尽管PSNR并不总是能反映这一点。

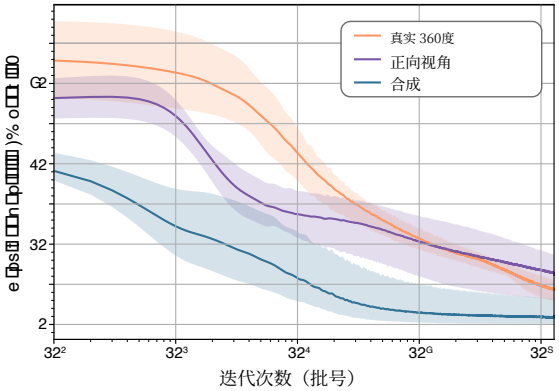


图 4. **梯度稀疏性**。 在最初的 12800 个批次（对于合成场景，一个 epoch）内，梯度在空间上变得非常稀疏，在合成情况下，每个批次只有 1% 的体素会更新。这通过稀疏参数更新实现了高效的训练。实线显示了平均值，而阴影区域显示了每种类型所有场景的完整数值范围。

正向视角场景，我们使用标准化设备坐标，如原始NeRF论文中定义的那样 [28]。

背景模型。 对于 360° 场景，我们使用多球面图像（MSI）背景模型来增强我们的稀疏体素网格前景表示，该模型也使用学习到的体素颜色和密度，并在球体内部和球体之间进行三线性插值。请注意，这与我们的前景模型基本相同，只是体素被使用简单的等矩形投影（体素索引覆盖球面角度 θ 和 ϕ）扭曲成球体。我们将64个球体沿逆半径线性放置，从1到∞（我们将内部场景预先缩放到大约包含在单位球内）。为了节省内存，我们仅存储颜色的rgb通道（仅零阶SH），并通过使用密度阈值化（如我们主模型中那样）稀疏存储所有层。这类似于NeRF++ [60]中的背景模型。

3.7. 正则化

我们通过图3展示了TV正则化的重要性。除了TV正则化（它鼓励平滑性并应用于所有场景），对于某些类型的场景，我们还使用额外的正则化器。3。

对于真实场景、正向视角和 360° 场景，我们使用基于SNeRG [10]的Cauchy损失的稀疏性先验：

$$\mathcal{L}_s = \lambda_s \sum_{i,k} \log\left(1 + 2\sigma(\mathbf{r}_i(t_k))^2\right) \qquad (4)$$

其中 σ（$\mathbf{r}_i$（t_k））表示训练射线 i上样本 k 的密度。在正向视角场景的优化每个小批量中，我们在每个活动射线上每个样本处评估该损失项。这也类似于PlenOctrees [59]中使用的稀疏性损失，并鼓励体素为空，这有助于节省内存。

说实话，在真实场景中，我们也在每个小批量中，对每条射线累积的前景透射率应用贝塔分布正则化器。这个损失项，遵循神经体积 [22]，，通过鼓励前景要么是完全不透明要么是空的，来促进清晰的前景-背景分解。

这个贝塔损失是：

$$\mathcal{L}_\beta = \lambda_\beta \sum_{\mathbf{r}} \left(\log(T_{FG}(\mathbf{r})) + \log(1 - T_{FG}(\mathbf{r}))\right) \qquad (5)$$

其中 $\mathbf{r}$ 是训练光线， $T_{FG}(\mathbf{r})$ 是光线 $\mathbf{r}$ 的累积前景透射率（介于 0 和 1 之间）。

3.8. 实现

由于稀疏体素体积渲染在现代自动微分库中支持不佳，我们创建了一个自定义的 PyTorch CUDA [31]扩展库来实现快速可微分的体积渲染。我们还提供了一个较慢、更高级的 JAX [5]实现。我们实现的 tốc độ 在很大程度上是因为我们的 Plenoxel 模型的梯度变得非常稀疏，如图 4所示。在优化的前 1-2 分钟内，不到 10% 的体素具有非零梯度。

4. 结果

我们展示了在合成、有界场景；真实、无界、正向视角场景；以及真实、无界 360°场景上的结果。我们包含了与先前工作的计时试验比较，显示在训练速度上相比所有先前方法有显著提升（同时支持实时渲染）。定量比较在表2中呈现，视觉比较在图1、图6、图7、图8中展示。我们的方法在优化后的第一个epoch就能达到高质量结果，耗时不到1.5分钟，如图5所示。2，1，6，7，8。

我们还展示了我们方法的各种消融实验结果。在正文中，我们展示了每种类型场景中所有场景的平均结果（PSNR、SSIM [56]，和VGG LPIPS [61]）；每个场景的完整定量和可视化结果，以及完整的实验细节（超参数等）都包含在补充材料中。



图5。 **1分钟，20秒　　ds**。 在优化1个epoch后，合成场景的结果，平均需要1分钟20秒。

	PSNR ↑	SSIM ↑	LPIPS ↓	训练时间
Ours	31.71	**0.958**	**0.049**	**11分钟**
NV [22]	26.05	0.893	0.160	>1 天
JAXNeRF [8, 28]	**31.85**	0.954	0.072	1.45 天
Ours	26.29	**0.839**	**0.210**	**24 分钟**
LLFF [27]	24.13	0.798	0.212	—*
JAXNeRF [8, 28]	**26.71**	0.820	0.235	1.62 天
Ours	20.40	**0.696**	**0.420**	**27分钟**
NeRF++ [60]	**20.49**	0.648	0.478	~4 天

表2。**结果**。 顶部：NeRF生成的8个合成场景的平均值； 中间：NeRF生成的8个真实正向视角场景； 底部：来自Tanks and Temples [16]的4个真实 360° 场景。其中4个合成场景训练时间少于10分钟。*LLFF需要预先训练网络以预测每个视点的MPI，然后才能在不进一步训练的情况下渲染新场景；这种预训练是分摊到所有场景中的，因此我们没有将其包含在表格中。

每个场景的完整实验细节（超参数等）都包含在补充材料中。

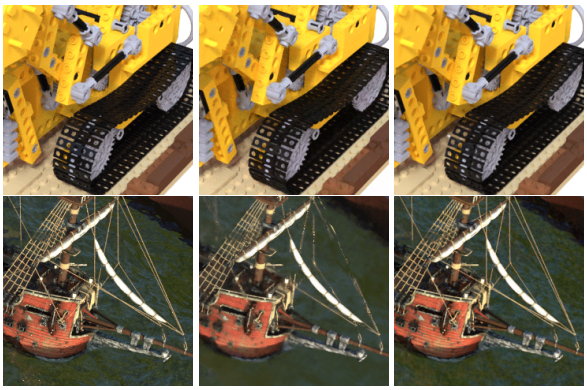
4.1. 合成场景

我们的合成实验使用了NeRF提供的8个场景：椅子、鼓、橄榄树、热狗、乐高、材料、麦克风和船。每个场景包含100个具有分辨率的真实值训练视点，这些视点的相机位置随机分布在面向物体的上半球内。每个场景在200个具有分辨率且相机位置已知的上半球内向内视点上进行评估。我们在表。2中提供定量比较，在图。6中提供视觉比较。。

我们将我们的方法与神经体积（NV） [22], 进行比较，与基于网格的3D卷积网络方法以及JAXNeRF [8, 28]。对于神经体积，我们使用 [28]中报告的值；对于JAXNeRF，我们报告了我们重新运行的结果，修复了其中心像素错误 [3]。我们的方法在最佳基线质量上达到了可比的水平，同时在单个GPU上平均每场景训练11分钟，并支持交互式渲染。

4.2. 正向视角真实场景

我们通过使用归一化设备坐标（NDC）将方法扩展到无界、正向视角的场景，即



真实值　　JAXNeRF [8, 28]　　Plenoxels

图6。**合成、有界场景**。NeRF在乐高和飞船合成场景上的示例结果 [28]。

在NeRF中推导出 [28]。我们的方法与我们在有界、合成场景上使用的版本基本相同，不同之处在于我们在整个优化过程中使用TV正则化（权重更强）。这种改变可能是必要的，因为根据第4.4节所述，这些场景的训练视点数量减少，

我们的正向视角实验使用了与 NeRF 相同的 8 个场景，其中 5 个最初来自 LLFF [27]。每个场景由 20 到 60 张正向视角图像组成，这些图像由分辨率 1008×756 的手持手机拍摄，其中 $\frac{7}{8}$ 的图像用于训练，其余的 $\frac{1}{8}$ 的图像被保留为测试集。

我们将我们的方法与局部光场融合 (LLFF) [27], 一种使用 3D 卷积网络为每个输入视图预测网格的先验方法以及 JAXNeRF 进行比较。我们在表 2 和图 7. 中提供了定量和视觉比较。

4.3. 真实 360° 场景

我们通过用多球体图像 (MSI，基于由 [62] 引入的多平面图像) 背景模型包围我们的稀疏体素网格，将我们的方法扩展到真实、无界的 360° 场景，其中每个背景球体也是一个简单的体素网格，具有三线性插值（在每个球体内部以及相邻层之间）。

我们的 360° 实验使用了坦克与寺庙数据集 [16] 中的 4个场景：M60、游乐场、火车和卡车。对于



真实值　　JAXNeRF [8, 28]　　Plenoxels

图7。 **真实、正向视角的场景**。 NeRF中蕨类和兰花正向视角场景的示例结果。

每个场景，我们使用了与 [39]相同的训练/测试分割。

我们将我们的方法与NeRF++ [60]进行比较，该模型通过添加背景模型来增强NeRF以表示无界场景。我们在表2中展示了定量比较，在图8中展示了视觉比较。

4.4. 消融研究

在本节中，我们对我们的方法进行广泛的消融研究，以了解哪些特征是其成功的关键，尤其是在这样一个简单的模型中。在表。1中，我们表明连续（在我们的案例中，三线性）插值与最近邻插值（即。每个体素内恒定）相比，显著提高了保真度。 [59]。

在表。3中，我们考虑我们的方法如何处理训练数据的急剧减少，从100个视点减少到25个视点，在8个合成场景上。我们将我们的方法与NeRF进行比较，发现尽管它缺乏复杂的神经先验，但通过增加TV正则化，我们的方法即使在有限的数据情况下也能优于NeRF。这次消融研究也阐明了为什么我们的模型在真实正向视角场景上与合成场景相比，在更高的TV正则化下表现更好：真实场景的训练图像很少，而更强的正则化有助于我们的优化平滑地扩展到稀疏监督区域。

我们还消融了表4中我们Plenoxel网格的分辨率4以及表5中的渲染公式5。来自Max的渲染公式与神经体积的相比产生了显著的改进 [22], 也许是因为它更符合物理规律（如补充材料中进一步讨论的那样）。补充材料还包括对学习率调度和优化器的消融实验，展示了Plenoxel优化对这些超参数的鲁棒性。

5. 讨论

我们提出了一种用于光逼真场景建模和全新视角渲染的方法，该方法产生的结果在保真度上可与当前最先进技术相媲美，同时训练时间却减少了几个数量级。我们的方法还

	PSNR ↑	SSIM ↑	LPIPS ↓
我们：100张图像（低TV）	31.71	0.958	0.050
NeRF：100张图像 [28]	31.01	0.947	0.081
我们：25张图像（低TV）	26.88	0.911	0.099
我们：25图像（高TV）	28.25	0.932	0.078
NeRF：25图像 [28]	27.78	0.925	0.108

表3。 **消融在视点数量上**。 通过增加我们的TV正则化，即使训练视点数量仅为完整数据集的四分之一，我们的保真度也超过了NeRF。结果是在NeRF的8个合成场景上取平均。

分辨率	PSNR ↑	SSIM ↑	LPIPS ↓
512^3	31.71	0.958	0.050
256^3	30.57	0.950	0.065
128^3	28.46	0.926	0.100
64^3	26.11	0.892	0.139
32^3	23.49	0.859	0.174

表4。**消融对plenoxel网格分辨率的影响**。结果是在NeRF生成的8个合成场景上取平均值。

渲染公式	PSNR ↑	SSIM ↑	LPIPS ↓
Max [24],用于NeRF [28]	30.57	0.950	0.065
神经体积 [22]	27.54	0.906	0.201

表 5. **不同渲染公式的比较**。 我们将 Max [24] （用于 NeRF 和我们的主要方法）的渲染公式与神经体积 [22], 中使用的渲染公式进行比较，后者使用绝对透射率而非相对透射率。结果是在 NeRF 的 8 个合成场景上平均得出的。

极其简洁明了，揭示了求解三维逆问题的核心要素：可微分的正向模型、连续表示（在我们的案例中，通过三线性插值实现）以及适当的正则化。我们承认，该方法所需的要素早已存在，然而具有数千万变量的非线性优化直到最近才成为计算机视觉从业者能够触及的技术。

局限性与未来工作。 与任何未定逆问题一样，我们的方法容易产生伪影。我们的方法与神经方法产生的伪影不同，如图9 所示，但两种方法在标准指标方面（如第4 节所述）都达到了相似的质量。未来的工作可能通过研究不同的正则化先验和/或更符合物理的微分渲染函数来调整或减轻这些剩余的伪影。

尽管我们为每个数据集都使用固定的超参数报告所有结果，但TV权重的最优预设值并不存在



真实值 NeRF++ [60] Plenoxels

图8。 **真实，** 360° **场景**。在游乐场和Tanks and Temples [1中的卡车 360° 场景上的示例结果 6].



真实值 JAXNeRF [8, 28] Plenoxels

图 9。 **伪影**。 JAXNeRF 和 Plenoxel 表现出略微不同的伪影，如图所示，在合成鼓场景中的高光部分。请注意，对于任何未确定的逆问题，某些伪影都是不可避免的，但具体的伪影会根据模型和正则化器引入的先验而变化。

更好的结果可以通过逐场景调整此参数获得，这得益于我们极快的训练时间。这很正常，因为不同场景的尺度、平滑度以及训练视点数量各不相同

我们的方法应该自然地扩展以支持多尺度渲染，并通过体素锥体追踪实现适当的抗锯齿，类似于Mip-NeRF中的修改 [3]。另一个简单的补充是色调映射，以解释白平衡和曝光变化 [42], 我们预计这将特别有助于真实 360° 场景。一个分层数据结构（例如八叉树）可能比我们的稀疏数组实现提供额外的加速，前提是保持可微插值。

由于我们的方法比 NeRF 快两个数量级，我们认为它可能使当前受 NeRF 性能瓶颈制约的下游应用得以实现——例如，多bounce光照和跨大型场景数据库的 3D 生成模型。通过将我们的方法与相机优化和大规模体素哈希等附加组件相结合，它可能实现一个端到端的逼真 3D 重建实用流程。

致谢

我们注意到 Utkarsh Singhal 和 Sara Fridovich-Keil 之前曾尝试用点云实现相关想法。此外，我们还要感谢 Ren Ng 提供的有益建议，以及 Hang Gao 审阅论文草稿。该项目部分由 DARPA 赞助的 CONIXResearch 中心、Google 研究人员奖（Angjoo Kanazawa）、Benjamin Recht 的 ONR 奖项（N00014-20-1-2497 和 N00014-18-1-2833）、NSF CPS 奖（1931853）以及 DARPA 确保自主性计划（FA8750-18-C-0101）提供慷慨支持。SFK 和 MT 由 NSF GRFP 资助。

参考文献

[1] Edward H. Adelson 和 James R. Bergen. 全视函数与早期视觉要素. 载于《视觉计算模型》，第 3–20 页. MIT 出版社, 1991. 2[2] Kara-Ali Aliev, Artem Sevastopolsky, Maria Kolos, Dmitry Ulyanov, 和 Victor Lempitsky. 基于神经网络的点云图形学. 载于《计算机视觉–ECCV 2020：第 16 届欧洲会议，英国格拉斯哥，2020 年 8 月 23–28 日，论文集，第 XXII 部分，第 696–712 页. 斯普林格出版社, 2020. 2[3] Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, 和 Pratul P. Srinivasan. Mip-NeRF：用于抗锯齿神经辐射场的多尺度表示，2021. 2, 4, 6, 8, 1[4] R. Basri 和 D.W. Jacobs. 拉曼反射和线性子空间. IEEE 模式分析与机器智能汇刊, 25(2):218–233, 2003. 2[5] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, J ake VanderPlas, Skye Wanderman-Milne, 和 Qiao Zhang. JAX：Python 程序的可组合变换, 2018. 2, 5+ Zhiqin Chen 和 Hao Zhang. 用于生成式形状建模的隐式场学习, 2019. 2[6] Christopher B. Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, 和 Silvio Savarese. 3d-r2n2：用于单视图和多视图 3D 对象重建的统一方法, 2016. 2[7] Boyang Deng, Jonathan T. Barron, 和 Pratul P. Srinivasan. JaxNeRF：NeRF 的 JAX 高效实现, 2020. 2, 4, 6, 7, 8, 5, 9, 10[8] Stephan J. Garbin, Marek Kowalski, Matthew Johnson, Jamie Shotton, 和 Julien Valentin. Fastnerf：200fps 的高保真神经渲染, 2021. 1, 2[9] Peter Hedman, Pratul P. Srinivasan, Ben Mildenhall, Jonathan T. Barron, 和 Paul Debevec. 用于实时视图合成的神经辐射场烘焙, 2021. 1, 2, 5[10]Geoffrey Hinton. RMSProp. 4, 1, 2[11] Christian H[12] ane, Shubham Tulsiani, 和 Jitendra Malik. 用于 3D 对象重建的层次表面预测, 2017. 2¨James T. Kajiya 和 Brian P Von Herzen. 光线追踪体积密度. 载于《第 11 届计算机图形学与交互技术年会的论文集，SIGGRAPH ’84》，第 165–174 页, 美国纽约, 1984. ACM. 2, 3[13] Abhishek Kar, Christian H[14] ane, 和 Jitendra Malik. 多视图立体机器学习, 2017. 2¨Petr Kellnhofer, Lars Jebe, Andrew Jones, Ryan Spicer, Kari Pulli, 和 Gordon Wetzstein. 神经光栅渲染, 2021. 2[15] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, 和 Vladlen Koltun. 坦克与寺庙：大规模场景重建基准测试. ACM 图形学汇刊, 36(4), 2017 年 7 月. 6, 8, 4[16] Aaron Knoll. 八叉树体积渲染方法综述, 2006. 2[17] Christoph Lassner 和 Michael Zollh[18] ofer. Pulsar：基于球体的神经渲染高效方法. 载于 IEEE/CVF 会议论文集计算机视觉与模式识别（CVPR），2021年6月。2[19] M. Levoy。体积数据的表面显示。IEEE计算机图形学与应用，8(3):29–37，1988。2[20] David B. Lindell、Julien N. P. Martel和Gordon Wetzstein。Autoint：快速神经体积渲染的自动集成，2021。2[21] Lingjie Liu、Jiatao Gu、Kyaw Zaw Lin、Tat-Seng Chua和Christian Theobalt。神经稀疏体素场，2021。1，2[22] Stephen Lombardi、Tomas Simon、Jason Saragih、Gabriel Schwartz、Andreas Lehrmann和Yaser Sheikh。神经体积。ACM图形学汇刊，38(4):1–14，2019年7月。2，5，6，7，3[23] Julien N. P. Martel、David B. Lindell、Connor Z. Lin、Eric R. Chan、Marco Monteiro和Gordon Wetzstein。Acorn：用于神经场景表示的自适应坐标网络，2021。2[24] N. Max。直接体积渲染的光学模型。IEEE可视化与计算机图形学汇刊，1(2):99–108，1995。2，3，7[25] Duane Merrill和NVIDIA公司。CUB：CUDA C++的协作原语，2021。1[26] Lars Mescheder、Michael Oechsle、Michael Niemeyer、Sebastian Nowozin和Andreas Geiger。占用网络：在函数空间中学习3D重建，2019。2[27] Ben Mildenhall、Pratul P. Srinivasan、Rodrigo Ortiz-Cayon、Nima Khademi Kalantari、Ravi Ramamoorthi、Ren Ng和Abhishek Kar。局部光场融合：具有规定采样指南的实用视图合成，2019。2，6，8[28] Ben Mildenhall、Pratul P Srinivasan、Matthew Tancik、Jonathan T Barron、Ravi Ramamoorthi和Ren Ng。Nerf：将场景表示为神经辐射场以进行视图合成。在欧几里得计算机视觉会议，第405–421页。Springer，2020。1，2，3，5，6，7，8，9，10[29] T. Neff、P. Stadlbauer、M. Parger、A. Kurz、J. H. Mueller、C. R. A. Chaitanya、A. Kaplanyan和M. Steinberger。Don- erf：使用深度预言网络实现紧凑型神经辐射场的实时渲染。计算机图形学论坛，40(4):45–59，2021年7月。2[30] Jorge Nocedal和Stephen J. Wright。数值优化。Springer，纽约，纽约，美国，第二版，2006。4[31] NVIDIA，P´eter Vingelmann和Frank H.P. Fitzek。CUDA，版本：10.2.89，2020。1，5[32] Alan V. Oppenheim和Ronald W. Schafer。离散时间信号处理。Prentice Hall出版社，美国，第三版，2009。2[33] Jeong Joon Park、Peter Florence、Julian Straub、Richard New- combe和Steven Lovegrove。DeepSDF：学习用于形状表示的连续符号距离函数，2019。2[34] Eric Penner和Li Zhang。软3D重建用于视图合成。ACM图形学汇刊（TOG），36 11，2017 1 2： – 。[35] Martin Piala和Ronald Clark。TermiNeRF：射线终止预测以实现高效的神经渲染，2021。2

[36] Ravi Ramamoorthi和Pat Hanrahan。关于辐射与辐照度之间的关系：从凸朗伯物体的图像确定照明。JOSA A，18(10):2448–2459，2001。2[37] Daniel Rebain、Wei Jiang、Soroosh Yazdani、Ke Li、Kwang Moo Yi和Andrea Tagliasacchi。Derf：分解辐射场，2020。1，2[38] Christian Reiser、Songyou Peng、Yiyi Liao和Andreas Geiger。KiloNeRF：使用数千个微小的MLPs加速神经辐射场，2021。1，2[39] Gernot Riegler和Vladlen Koltun。自由视图合成，2020。7[40] Gernot Riegler、Ali Osman Ulusoy和Andreas Geiger。Oct- net：学习高分辨率深度3D表示，2017。2[41] Leonid I Rudin和Stanley Osher。基于总变分的图像恢复与自由局部约束。在第一届图像处理国际会议论文集，卷1，第31–35页。IEEE，1994。4[42] Darius R¨uckert、Linus Franke和Marc Stamminger。Adop：近似可微分的单像素点渲染，2021。2，8[43] Seitz、Steven、Kutulakos和Kiriakos。空间雕刻的形状理论。01 2000。2[44] S.M. Seitz和C.R. Dyer。基于体素着色的逼真场景重建。在IEEE计算机学会计算机视觉与模式识别会议论文集，第1067–1073页，1997。2[45] Vincent Sitzmann、Justus Thies、Felix Heide、Matthias Nießner、Gordon Wetzstein和Michael Zollh¨ofer。Deepvox- els：学习持久的3D特征嵌入，2019。2[46] Vincent Sitzmann、Michael Zollh¨ofer和Gordon Wetzstein。场景表示网络：连续3D结构感知的神经场景表示，2020。2[47] Peter-Pike Sloan、Jan Kautz和John Snyder。动态、低频照明环境中的预计算辐射传输，用于实时渲染。ACM Trans. Graph.，21(3):527–536，2002年7月。2[48] Pratul P. Srinivasan、Richard Tucker、Jonathan T. Barron、Ravi Ramamoorthi、Ren Ng和Noah Snavely。使用多平面图像扩展视图边界，2019。2[49] Richard Szeliski和Polina Golland。具有透明度和贴边的立体匹配。国际计算机视觉杂志，32:45–61，2004。2[50] Towaki Takikawa、Joey Litalien、Kangxue Yin、Karsten Kreis、Charles Loop、Derek Nowrouzezahrai、Alec Jacobson、Mor- gan McGuire和Sanja Fidler。神经几何细节层次：使用隐式3D形状实现实时渲染，2021。2[51] Matthew Tancik、Ben Mildenhall、Terrance Wang、Divi Schmidt、Pratul P. Srinivasan、Jonathan T. Barron和Ren Ng。基于坐标的神经表示的优化初始化，2021。2[52] Maxim Tatarchenko、Alexey Dosovitskiy和Thomas Brox。八叉树生成网络：用于高分辨率3D输出的高效卷积架构，2017。2[53] Shubham Tulsiani、Tinghui Zhou、Alexei A. Efros 和Jiten-dra Malik。基于可微分射线一致性的多视角监督单视角重建，2017年。2[54] Craig Upson 和 Michael Keeler。V-buffer：可见体积渲染。SIGGRAPH 计算图形，22(4):59–64，1988年6月。2[55] Peng-Shuai Wang、Yang Liu、Yu-Xiao Guo、Chun-Yu Sun 和 Xin Tong。O-cnn。ACM 图形学汇刊，36(4):1–11，2017年7月。2[56] Zhou Wang、A.C. Bovik、H.R. Sheikh 和 E.P. Simon-celli。图像质量评估：从错误可见性到结构相似性。IEEE 图像处理汇刊，13(4):600–612，2004年。5[57] Olivia Wiles、Georgia Gkioxari、Richard Szeliski 和 Justin Johnson。Synsin：从单张图像端到端生成视图，2020年。2[58] Suttisak Wizadwongsa、Pakkapon Phongthawee、Jiraphon Yenphraphai 和 Supasorn Suwajanakorn。Nex：基于神经基函数扩展的实时视图合成，2021年。2[59] Alex Yu、Ruilong Li、Matthew Tancik、Hao Li、Ren Ng 和Angjoo Kanazawa。用于实时渲染神经辐射场的PlenOctrees。在ICCV上，2021年。1、2、3、4、5、7[60] Kai Zhang、Gernot Riegler、Noah Snavely 和 Vladlen Koltun。NeRF++：分析和改进神经辐射场，2020年。1、2、5、6、7、8、4、11[61] Richard Zhang、Phillip Isola、Alexei A Efros、Eli Shechtman 和 Oliver Wang。深度特征作为感知度量的不合理有效性。在CVPR上，2018年。5[62] Tinghui Zhou、Richard Tucker、John Flynn、Graham Fyffe 和 Noah Snavely。立体放大：使用多平面图像学习视图合成，2018年。2、6